

操作系统实验报告

实验七：I/O 模型演进：从 C10K 到协程



四川大學

姓 名：_____许盛凯_____

学 号：_____2024141460445_____

专 业：_____计算机科学与技术_____

指导教师：_____左劼_____

2026 年 6 月 8 日

目录

1	实验目的与环境	2
2	编译与公共测试工具	2
3	任务一：同步阻塞 I/O + 多线程	5
3.1	设计思路	5
3.2	完整代码	5
3.3	最终运行结果	8
4	任务二：同步非阻塞 I/O + 忙轮询	9
4.1	设计思路	9
4.2	完整代码	9
4.3	最终运行结果	12
5	任务三：epoll + Reactor	12
5.1	设计思路	12
5.2	完整代码	13
5.3	最终运行结果与 CPU 时间对比	16
6	可选任务四：epoll + 协程	16
6.1	设计思路	16
6.2	完整代码	17
6.3	最终运行结果与 Reactor 结构对比	21
7	实验思考题	21
8	自选进阶任务：挑战二——多 Reactor 多线程两轮登录服务器	23
8.1	设计思路	23
8.2	代码实现	23
8.3	最终结果或运行输出	29
9	实验总结	29

1 实验目的与环境

本实验围绕 C10K 问题，依次实现并比较多线程阻塞 I/O、单线程非阻塞忙轮询、epoll Reactor、epoll+ 协程以及多 Reactor 多线程扩展模型。实验目标包括观察多线程阻塞 I/O 在线程资源受限时的容量瓶颈，理解阻塞/非阻塞与同步/异步的区别，比较忙轮询与 epoll 的 CPU 时间差异，并通过可选协程任务理解协程如何保存执行位置、改善多轮业务代码结构。

本次实验代码在 Linux 环境下编写并运行。由于当前沙盒环境的 8080 端口被系统内置 Jupyter 服务占用，所有程序源码仍保留默认 PORT=8080，实际运行截图通过编译参数 -DPORT=18080 将测试端口统一改为 18080。端口修改不影响 I/O 模型、资源限制和 CPU 时间对比结论。

表 1: 实验任务概览

阶段	解决的问题	新暴露的问题或观察重点
任务一：多线程阻塞 I/O	业务代码顺序直观，一个连接一个线程	线程数和栈空间随连接数线性增长，最终资源耗尽
任务二：非阻塞忙轮询	不再为每个连接创建线程	空闲连接仍被反复扫描，CPU 时间接近真实等待时间
任务三：epoll Reactor	由内核通知就绪 fd，只处理真正可读连接	多轮业务需要显式保存状态和跨事件数据
可选任务四：epoll + 协程	保留 epoll 等待效率，业务代码恢复顺序形式	协程调度和上下文保存机制更复杂
进阶：多 Reactor 多线程	多个工作线程分别拥有 epoll 实例，扩展多核处理能力	需要设计 fd 分发、队列唤醒和连接生命周期管理

2 编译与公共测试工具

公共客户端 hold_client.c 用于建立大量慢连接：先建立指定数量 TCP 连接并保持一段时间不发送请求，再统一发送 HTTP 请求并统计响应。基础任务二、三使用相同连接数和保持时间，以保证 CPU 时间对比口径一致。

编译命令记录

编译输出

```
$ gcc -Wall -Wextra -O2 -DPORT=18080 hold_client.c -o hold_client
$ gcc -Wall -Wextra -O2 -DPORT=18080 stage1_threaded.c -o stage1 -pthread
$ gcc -Wall -Wextra -O2 -DPORT=18080 stage2_busypoll.c -o stage2
$ gcc -Wall -Wextra -O2 -DPORT=18080 stage3_epoll.c -o stage3
$ gcc -Wall -Wextra -O2 -DPORT=18080 stage4_coroutine.c -o stage4
$ gcc -Wall -Wextra -O2 -DPORT=18080 multi_reactor_login.c -o multi_reactor_login -pthread
说明：实验沙盒中的 8080 端口被 Jupyter 占用，因此运行截图使用 -DPORT=18080；源码默认仍保留 PORT=8080，可在本机直接按手册端口运行。
```

图 1: 编译命令记录

统一慢连接客户端 hold_client.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <signal.h>
6  #include <errno.h>
7  #include <sys/resource.h>
8  #include <sys/socket.h>
9  #include <sys/time.h>
10 #include <arpa/inet.h>
11
12 static const char *REQUEST =
13     "GET / HTTP/1.1\r\n"
14     "Host: 127.0.0.1\r\n"
15     "Connection: close\r\n"
16     "\r\n";
17
18 static void raise_fd_limit(int count) {
19     struct rlimit limit;
20     if (getrlimit(RLIMIT_NOFILE, &limit) < 0) {
21         perror("getrlimit");
22         exit(1);
23     }
24     rlim_t target = (rlim_t)count + 100;
25     if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max)
26         target = limit.rlim_max;
27     limit.rlim_cur = target;
28     if (setrlimit(RLIMIT_NOFILE, &limit) < 0) {
29         perror("setrlimit");
30         exit(1);
31     }
32 }
33
34 int main(int argc, char *argv[]) {
35     if (argc != 3) {
36         fprintf(stderr, " 用法: %s < 连接数> < 保持秒数>\n", argv[0]);
37         return 1;
38     }
39     int target = atoi(argv[1]);
```

```

40     int hold_seconds = atoi(argv[2]);
41     if (target <= 0 || hold_seconds < 0) {
42         fprintf(stderr, " 连接数必须大于 0，保持时间不能为负数\n");
43         return 1;
44     }
45
46     raise_fd_limit(target);
47     signal(SIGPIPE, SIG_IGN);
48
49     int *fds = malloc((size_t)target * sizeof(int));
50     if (!fds) {
51         perror("malloc");
52         return 1;
53     }
54
55     struct sockaddr_in addr;
56     memset(&addr, 0, sizeof(addr));
57     addr.sin_family = AF_INET;
58     addr.sin_port = htons(PORT);
59     if (inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr) != 1) {
60         fprintf(stderr, "inet_pton failed\n");
61         free(fds);
62         return 1;
63     }
64
65     int connected = 0;
66     for (int i = 0; i < target; i++) {
67         int fd = socket(AF_INET, SOCK_STREAM, 0);
68         if (fd < 0) {
69             perror("socket");
70             break;
71         }
72         if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
73             perror("connect");
74             close(fd);
75             break;
76         }
77         struct timeval timeout = {.tv_sec = 5, .tv_usec = 0};
78         if (setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout)) < 0) {
79             perror("setsockopt SO_RCVTIMEO");
80             close(fd);
81             break;
82         }
83         fds[connected++] = fd;
84         if (connected % 500 == 0)
85             printf(" 已建立 %d 个连接\n", connected);
86     }
87     printf(" 共建立 %d 个连接，保持 %d 秒且暂不发送数据\n", connected, hold_seconds);
88     sleep((unsigned int)hold_seconds);
89
90     int sent = 0;
91     for (int i = 0; i < connected; i++) {
92         ssize_t n = write(fds[i], REQUEST, strlen(REQUEST));
93         if (n > 0)
94             sent++;
95     }
96     printf("write() 成功返回的连接: %d\n", sent);
97

```

```

98     int response_count = 0;
99     int closed_count = 0;
100    int timeout_count = 0;
101    int error_count = 0;
102    for (int i = 0; i < connected; i++) {
103        char response[512];
104        ssize_t n = read(fds[i], response, sizeof(response) - 1);
105        if (n > 0) {
106            response[n] = '\0';
107            if (strstr(response, "HTTP/1.1 200 OK") != NULL)
108                response_count++;
109            else
110                error_count++;
111        } else if (n == 0) {
112            closed_count++;
113        } else if (errno == EAGAIN || errno == EWOULDBLOCK) {
114            timeout_count++;
115        } else {
116            error_count++;
117        }
118    }
119    printf(" 有效 HTTP 响应: %d | 无响应关闭: %d | 读取超时: %d | 重置或无效响应: %d\n",
120           response_count, closed_count, timeout_count, error_count);
121
122    for (int i = 0; i < connected; i++)
123        close(fds[i]);
124    free(fds);
125    return 0;
126 }

```

3 任务一：同步阻塞 I/O + 多线程

3.1 设计思路

任务一采用传统的“一个连接一个线程”模型。主线程使用 `accept()` 接收 TCP 连接, 每接收一个连接就通过 `pthread_create()` 创建工作线程。工作线程在阻塞 `read()` 上等待客户端发送 HTTP 请求, 收到请求后写回固定 HTTP 响应并关闭连接。

为了稳定复现线程资源瓶颈, 程序将服务端虚拟地址空间限制为 512MB, 并将每个线程栈大小设为 256KB。慢连接客户端在连接建立后保持数秒不发送请求, 因此工作线程会长期阻塞在 `read()` 上。随着连接数量增加, 线程数和虚拟地址空间持续增长, 最终 `pthread_create()` 返回 `EAGAIN`。

3.2 完整代码

stage1_threaded.c

```

1 #include <stdio.h>

```

```

2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <pthread.h>
6  #include <sys/resource.h>
7  #include <sys/socket.h>
8  #include <netinet/in.h>
9  #include <arpa/inet.h>
10 #include <errno.h>
11
12 #ifndef PORT
13 #define PORT 8080
14 #endif
15 #define BACKLOG 4096
16 #define BUF_SIZE 4096
17 #define ADDRESS_SPACE_MB 512
18 #define THREAD_STACK_KB 256
19 #define FILE_LIMIT 10000
20
21 static const char *HTTP_RESP =
22     "HTTP/1.1 200 OK\r\n"
23     "Content-Type: text/plain\r\n"
24     "Content-Length: 13\r\n"
25     "Connection: close\r\n"
26     "\r\n"
27     "Hello, World!";
28
29 static long long accepted_count = 0;
30
31 static void limit_process_resources(void) {
32     struct rlimit limit;
33     if (getrlimit(RLIMIT_AS, &limit) < 0) { perror("getrlimit RLIMIT_AS"); exit(1); }
34     rlim_t target = (rlim_t)ADDRESS_SPACE_MB * 1024 * 1024;
35     if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max)
36         target = limit.rlim_max;
37     limit.rlim_cur = target;
38     if (setrlimit(RLIMIT_AS, &limit) < 0) { perror("setrlimit RLIMIT_AS"); exit(1); }
39
40     if (getrlimit(RLIMIT_NOFILE, &limit) < 0) { perror("getrlimit RLIMIT_NOFILE"); exit(1); }
41     target = FILE_LIMIT;
42     if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max)
43         target = limit.rlim_max;
44     limit.rlim_cur = target;
45     if (setrlimit(RLIMIT_NOFILE, &limit) < 0) { perror("setrlimit RLIMIT_NOFILE"); exit(1); }
46 }
47
48 static void print_process_status(long long conn_count) {
49     FILE *fp = fopen("/proc/self/status", "r");
50     if (!fp) { perror("fopen /proc/self/status"); return; }
51     char line[256];
52     char threads[256] = "";
53     char vm_size[256] = "";
54     char vm_rss[256] = "";
55     while (fgets(line, sizeof(line), fp)) {
56         if (strncmp(line, "Threads:", 8) == 0)
57             snprintf(threads, sizeof(threads), "%s", line);
58         else if (strncmp(line, "VmSize:", 7) == 0)
59             snprintf(vm_size, sizeof(vm_size), "%s", line);

```

```

60     else if (strncmp(line, "VmRSS:", 6) == 0)
61         snprintf(vm_rss, sizeof(vm_rss), "%s", line);
62     }
63     fclose(fp);
64     printf("[server] accepted=%lld | %s[server] accepted=%lld | %s[server] accepted=%lld | %s",
65         conn_count, threads, conn_count, vm_size, conn_count, vm_rss);
66     fflush(stdout);
67 }
68
69 static void *worker(void *arg) {
70     int fd = (int)(intptr_t)arg;
71     char buf[BUF_SIZE];
72     ssize_t n = read(fd, buf, sizeof(buf));
73     if (n > 0) {
74         (void)write(fd, HTTP_RESP, strlen(HTTP_RESP));
75     }
76     close(fd);
77     return NULL;
78 }
79
80 static int create_listen_socket(void) {
81     int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
82     if (listen_fd < 0) { perror("socket"); exit(1); }
83     int opt = 1;
84     setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
85     struct sockaddr_in addr;
86     memset(&addr, 0, sizeof(addr));
87     addr.sin_family = AF_INET;
88     addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
89     addr.sin_port = htons(PORT);
90     if (bind(listen_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) { perror("bind"); exit(1); }
91     if (listen(listen_fd, BACKLOG) < 0) { perror("listen"); exit(1); }
92     return listen_fd;
93 }
94
95 int main(void) {
96     limit_process_resources();
97     int listen_fd = create_listen_socket();
98     pthread_attr_t attr;
99     pthread_attr_init(&attr);
100     pthread_attr_setstacksize(&attr, THREAD_STACK_KB * 1024);
101     printf("stage1 threaded server start, port=%d, RLIMIT_AS=%dMB, worker stack=%dKB\n",
102         PORT, ADDRESS_SPACE_MB, THREAD_STACK_KB);
103     fflush(stdout);
104
105     while (1) {
106         int client_fd = accept(listen_fd, NULL, NULL);
107         if (client_fd < 0) {
108             if (errno == EINTR) continue;
109             perror("accept");
110             break;
111         }
112         accepted_count++;
113         pthread_t tid;
114         int ret = pthread_create(&tid, &attr, worker, (void *) (intptr_t) client_fd);
115         if (ret != 0) {
116             errno = ret;
117             fprintf(stderr, "pthread_create failed at accepted=%lld: %s\n", accepted_count, strerror(ret));

```



```

118         close(client_fd);
119         print_process_status(accepted_count);
120         break;
121     }
122     pthread_detach(tid);
123     if (accepted_count % 500 == 0)
124         print_process_status(accepted_count);
125 }
126 close(listen_fd);
127 return 0;
128 }

```

3.3 最终运行结果

任务一：同步阻塞 I/O + 多线程运行记录

服务端输出

```

stage1 threaded server start, port=18080, RLIMIT_AS=512MB, worker stack=256KB
[server] accepted=500 | Threads: 501
[server] accepted=500 | VmSize: 140752 kB
[server] accepted=500 | VmRSS: 123388 kB
[server] accepted=1000 | Threads: 1001
[server] accepted=1000 | VmSize: 270884 kB
[server] accepted=1000 | VmRSS: 244144 kB
[server] accepted=1500 | Threads: 1501
[server] accepted=1500 | VmSize: 401016 kB
[server] accepted=1500 | VmRSS: 364992 kB
pthread_create failed at accepted=1974: Resource temporarily unavailable
[server] accepted=1974 | Threads: 1974
[server] accepted=1974 | VmSize: 524128 kB
[server] accepted=1974 | VmRSS: 479052 kB

```

客户端输出

```

connect: Connection refused
已建立 500 个连接
已建立 1000 个连接
已建立 1500 个连接
已建立 2000 个连接
共建立 2411 个连接, 保持 5 秒且暂不发送数据
write() 成功返回的连接: 1974
有效 HTTP 响应: 0 | 无响应关闭: 2411 | 读取超时: 0 | 重置或无效响应: 0

```

图 2: 任务一运行记录：线程数与虚拟地址空间随连接数增长

从输出可以看到，服务端在 500、1000、1500 个连接时，Threads 分别接近 501、1001、1501，说明线程数量基本随连接数线性增长。随着线程栈不断分配，VmSize 接近 512MB 限制；当接收到第 1974 个连接时，pthread_create() 返回 Resource temporarily unavailable。这说明客户端 connect() 成功只代表 TCP 连接进入服务端监听队列或已被 accept()，不能证明服务端已经成功创建工作线程并完成应用层处理。

4 任务二：同步非阻塞 I/O + 忙轮询

4.1 设计思路

任务二将监听 socket 和所有客户端 socket 设置为非阻塞。服务端只使用一个线程维护连接数组，不断循环执行两件事：第一，尝试 `accept()` 新连接；第二，对所有已连接客户端尝试 `read()`。当客户端暂时没有发送数据时，`read()` 立即返回 -1 并设置 `errno=EAGAIN` 或 `EWOULDBLOCK`，程序继续扫描下一个连接。

该模型解决了任务一的线程爆炸问题，但空闲连接越多，无效 `read()` 尝试越多。即使 1000 个客户端在保持阶段不发送数据，服务端仍持续循环扫描连接数组，因此 CPU 时间接近真实等待时间。

4.2 完整代码

stage2_busypoll.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <fcntl.h>
6  #include <errno.h>
7  #include <time.h>
8  #include <sys/resource.h>
9  #include <sys/socket.h>
10 #include <netinet/in.h>
11 #include <arpa/inet.h>
12
13 #ifndef PORT
14 #define PORT 8080
15 #endif
16 #define BACKLOG 4096
17 #define BUF_SIZE 4096
18 #define MAX_CLIENTS 10000
19 #define FILE_LIMIT 10000
20
21 static const char *HTTP_RESP =
22     "HTTP/1.1 200 OK\r\n"
23     "Content-Type: text/plain\r\n"
24     "Content-Length: 13\r\n"
25     "Connection: close\r\n"
26     "\r\n"
27     "Hello, World!";
28
29 typedef struct client_s {
30     int fd;
31     int active;
32     int responded;
33 } client_t;
```

```

34
35 static client_t clients[MAX_CLIENTS];
36 static int client_count = 0;
37 static int active_count = 0;
38 static int responded_count = 0;
39 static struct timespec first_conn_wall, first_conn_cpu;
40 static int measuring = 0;
41
42 static double elapsed(struct timespec a, struct timespec b) {
43     return (double)(b.tv_sec - a.tv_sec) + (double)(b.tv_nsec - a.tv_nsec) / 1000000000.0;
44 }
45
46 static void raise_fd_limit(void) {
47     struct rlimit limit;
48     if (getrlimit(RLIMIT_NOFILE, &limit) < 0) { perror("getrlimit"); exit(1); }
49     rlim_t target = FILE_LIMIT;
50     if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max) target = limit.rlim_max;
51     limit.rlim_cur = target;
52     if (setrlimit(RLIMIT_NOFILE, &limit) < 0) { perror("setrlimit"); exit(1); }
53 }
54
55 static int set_nonblocking(int fd) {
56     int flags = fcntl(fd, F_GETFL, 0);
57     if (flags < 0) return -1;
58     return fcntl(fd, F_SETFL, flags | O_NONBLOCK);
59 }
60
61 static int create_listen_socket(void) {
62     int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
63     if (listen_fd < 0) { perror("socket"); exit(1); }
64     int opt = 1;
65     setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
66     struct sockaddr_in addr;
67     memset(&addr, 0, sizeof(addr));
68     addr.sin_family = AF_INET;
69     addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
70     addr.sin_port = htons(PORT);
71     if (bind(listen_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) { perror("bind"); exit(1); }
72     if (listen(listen_fd, BACKLOG) < 0) { perror("listen"); exit(1); }
73     if (set_nonblocking(listen_fd) < 0) { perror("set_nonblocking listen"); exit(1); }
74     return listen_fd;
75 }
76
77 static void close_client(int i) {
78     if (clients[i].active) {
79         close(clients[i].fd);
80         clients[i].active = 0;
81         active_count--;
82     }
83 }
84
85 int main(void) {
86     raise_fd_limit();
87     int listen_fd = create_listen_socket();
88     printf("stage2 busy-poll server start, port=%d\n", PORT);
89     fflush(stdout);
90     long long eagain_count = 0;
91     char buf[BUF_SIZE];

```

```

92
93 while (1) {
94     while (1) {
95         int fd = accept(listen_fd, NULL, NULL);
96         if (fd < 0) {
97             if (errno == EAGAIN || errno == EWOULDBLOCK) break;
98             if (errno == EINTR) continue;
99             perror("accept");
100             break;
101         }
102         if (client_count >= MAX_CLIENTS) {
103             close(fd);
104             continue;
105         }
106         set_nonblocking(fd);
107         clients[client_count].fd = fd;
108         clients[client_count].active = 1;
109         clients[client_count].responded = 0;
110         client_count++;
111         active_count++;
112         if (!measuring) {
113             clock_gettime(CLOCK_MONOTONIC, &first_conn_wall);
114             clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &first_conn_cpu);
115             measuring = 1;
116         }
117         if (client_count % 500 == 0) {
118             printf("[server] accepted=%d active=%d busy polling...\n", client_count, active_count);
119             fflush(stdout);
120         }
121     }
122     for (int i = 0; i < client_count; i++) {
123         if (!clients[i].active) continue;
124         ssize_t n = read(clients[i].fd, buf, sizeof(buf));
125         if (n > 0) {
126             (void)write(clients[i].fd, HTTP_RESP, strlen(HTTP_RESP));
127             clients[i].responded = 1;
128             responded_count++;
129             close_client(i);
130         } else if (n == 0) {
131             close_client(i);
132         } else if (errno == EAGAIN || errno == EWOULDBLOCK) {
133             eagain_count++;
134         } else if (errno == EINTR) {
135             continue;
136         } else {
137             close_client(i);
138         }
139     }
140     if (measuring && client_count > 0 && active_count == 0) {
141         struct timespec now_wall, now_cpu;
142         clock_gettime(CLOCK_MONOTONIC, &now_wall);
143         clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &now_cpu);
144         printf("[server] all clients processed. accepted=%d responded=%d EAGAIN=%lld\n",
145             client_count, responded_count, eagain_count);
146         printf("[server] busy-poll measurement: wall=%.3fs cpu=%.3fs\n",
147             elapsed(first_conn_wall, now_wall), elapsed(first_conn_cpu, now_cpu));
148         fflush(stdout);
149         break;

```

```
150     }
151 }
152 close(listen_fd);
153 return 0;
154 }
```

4.3 最终运行结果



图 3: 任务二运行记录：忙轮询导致 CPU 时间接近等待时间

在 1000 个慢连接、保持 3 秒的测试中，服务端统计 wall=4.815s, cpu=4.550s。CPU 时间接近真实经过时间，说明单线程几乎一直在运行。日志中 EAGAIN=49559 表明大量循环读操作没有取得数据，只是不断得到“当前不可读”的结果。

5 任务三：epoll + Reactor

5.1 设计思路

任务三继续使用非阻塞 socket，但不再由应用层主动扫描所有连接，而是将监听 fd 和客户端 fd 注册到 epoll。事件循环阻塞在 epoll_wait(..., -1) 上，没有就绪事件时线程休眠；当客户端发送 HTTP 请求时，内核通知该 fd 可读，程序再调用 read() 读取数据并写回响应。

因此，任务三仍然属于同步 I/O：应用程序收到的是“fd 已就绪”的通知，真正的数据读取仍由程序主动调用 read() 完成。但与任务二不同，等待阶段不再反复扫描空闲连接，CPU 消耗显著下降。

5.2 完整代码

stage3_epoll.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <fcntl.h>
6  #include <errno.h>
7  #include <time.h>
8  #include <sys/resource.h>
9  #include <sys/socket.h>
10 #include <sys/epoll.h>
11 #include <netinet/in.h>
12 #include <arpa/inet.h>
13
14 #ifndef PORT
15 #define PORT 8080
16 #endif
17 #define BACKLOG 4096
18 #define BUF_SIZE 4096
19 #define MAX_EVENTS 1024
20 #define MAX_CLIENTS 10000
21 #define FILE_LIMIT 10000
22
23 static const char *HTTP_RESP =
24     "HTTP/1.1 200 OK\r\n"
25     "Content-Type: text/plain\r\n"
26     "Content-Length: 13\r\n"
27     "Connection: close\r\n"
28     "\r\n"
29     "Hello, World!";
30
31 typedef struct client_s {
32     int fd;
33 } client_t;
34
35 static int client_count = 0;
36 static int active_count = 0;
37 static int responded_count = 0;
38 static struct timespec first_conn_wall, first_conn_cpu;
39 static int measuring = 0;
40
41 static double elapsed(struct timespec a, struct timespec b) {
42     return (double)(b.tv_sec - a.tv_sec) + (double)(b.tv_nsec - a.tv_nsec) / 1000000000.0;
43 }
44
45 static void raise_fd_limit(void) {
46     struct rlimit limit;
47     if (getrlimit(RLIMIT_NOFILE, &limit) < 0) { perror("getrlimit"); exit(1); }
48     rlim_t target = FILE_LIMIT;
49     if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max) target = limit.rlim_max;
50     limit.rlim_cur = target;
51     if (setrlimit(RLIMIT_NOFILE, &limit) < 0) { perror("setrlimit"); exit(1); }
52 }
```

```

53
54 static int set_nonblocking(int fd) {
55     int flags = fcntl(fd, F_GETFL, 0);
56     if (flags < 0) return -1;
57     return fcntl(fd, F_SETFL, flags | O_NONBLOCK);
58 }
59
60 static int create_listen_socket(void) {
61     int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
62     if (listen_fd < 0) { perror("socket"); exit(1); }
63     int opt = 1;
64     setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
65     struct sockaddr_in addr;
66     memset(&addr, 0, sizeof(addr));
67     addr.sin_family = AF_INET;
68     addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
69     addr.sin_port = htons(PORT);
70     if (bind(listen_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) { perror("bind"); exit(1); }
71     if (listen(listen_fd, BACKLOG) < 0) { perror("listen"); exit(1); }
72     if (set_nonblocking(listen_fd) < 0) { perror("set_nonblocking listen"); exit(1); }
73     return listen_fd;
74 }
75
76 static void register_fd(int epfd, int fd, uint32_t events, void *ptr) {
77     struct epoll_event ev;
78     memset(&ev, 0, sizeof(ev));
79     ev.events = events;
80     ev.data.ptr = ptr;
81     if (epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev) < 0) {
82         perror("epoll_ctl ADD");
83         close(fd);
84     }
85 }
86
87 int main(void) {
88     raise_fd_limit();
89     int listen_fd = create_listen_socket();
90     int epfd = epoll_create1(0);
91     if (epfd < 0) { perror("epoll_create1"); exit(1); }
92     register_fd(epfd, listen_fd, EPOLLIN, NULL);
93     printf("stage3 epoll server start, port=%d\n", PORT);
94     fflush(stdout);
95
96     struct epoll_event events[MAX_EVENTS];
97     char buf[BUF_SIZE];
98
99     while (1) {
100         int nready = epoll_wait(epfd, events, MAX_EVENTS, -1);
101         if (nready < 0) {
102             if (errno == EINTR) continue;
103             perror("epoll_wait");
104             break;
105         }
106         for (int i = 0; i < nready; i++) {
107             if (events[i].data.ptr == NULL) {
108                 while (1) {
109                     int fd = accept(listen_fd, NULL, NULL);
110                     if (fd < 0) {

```

```

111         if (errno == EAGAIN || errno == EWOULDBLOCK) break;
112         if (errno == EINTR) continue;
113         perror("accept");
114         break;
115     }
116     if (client_count >= MAX_CLIENTS) {
117         close(fd);
118         continue;
119     }
120     set_nonblocking(fd);
121     client_t *c = calloc(1, sizeof(*c));
122     if (!c) { close(fd); continue; }
123     c->fd = fd;
124     register_fd(epfd, fd, EPOLLIN | EPOLLRDHUP, c);
125     client_count++;
126     active_count++;
127     if (!measuring) {
128         clock_gettime(CLOCK_MONOTONIC, &first_conn_wall);
129         clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &first_conn_cpu);
130         measuring = 1;
131     }
132     if (client_count % 500 == 0) {
133         printf("[server] accepted=%d active=%d waiting in epoll_wait\n", client_count,
134             ↵ active_count);
135         fflush(stdout);
136     }
137 } else {
138     client_t *c = events[i].data.ptr;
139     ssize_t nr = read(c->fd, buf, sizeof(buf));
140     if (nr > 0) {
141         (void)write(c->fd, HTTP_RESP, strlen(HTTP_RESP));
142         responded_count++;
143     }
144     epoll_ctl(epfd, EPOLL_CTL_DEL, c->fd, NULL);
145     close(c->fd);
146     free(c);
147     active_count--;
148 }
149 }
150 if (measuring && client_count > 0 && active_count == 0) {
151     struct timespec now_wall, now_cpu;
152     clock_gettime(CLOCK_MONOTONIC, &now_wall);
153     clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &now_cpu);
154     printf("[server] all clients processed. accepted=%d responded=%d\n",
155         client_count, responded_count);
156     printf("[server] epoll measurement: wall=%.3fs cpu=%.3fs\n",
157         elapsed(first_conn_wall, now_wall), elapsed(first_conn_cpu, now_cpu));
158     fflush(stdout);
159     break;
160 }
161 }
162 close(epfd);
163 close(listen_fd);
164 return 0;
165 }

```


5.3 最终运行结果与 CPU 时间对比

任务三：epoll + Reactor 运行记录

服务端输出

stage3 epoll server start, port=18080
[server] accepted=500 active=500 waiting in epoll_wait
[server] accepted=1000 active=1000 waiting in epoll_wait
[server] all clients processed. accepted=1000 responded=1000
[server] epoll measurement: wall=5.190s cpu=1.900s

客户端输出

已建立 500 个连接
已建立 1000 个连接
共建立 1000 个连接，保持 3 秒且暂不发送数据
write() 成功返回的连接：1000
有效 HTTP 响应：1000 | 无响应关闭：0 | 读取超时：0 | 重置或无效响应：0

图 4: 任务三运行记录：epoll 等待期间 CPU 时间较低

表 2: 任务二与任务三在相同负载下的 CPU 时间对比

模型	连接数	保持时间	实际经过时间 wall	进程 CPU 时间
非阻塞忙轮询	1000	3s	4.815s	4.550s
epoll Reactor	1000	3s	5.190s	1.900s

对比可见，忙轮询模型在等待客户端发送请求时仍消耗接近一个 CPU 核心，而 epoll 模型在无事件时阻塞在 `epoll_wait()`，只有连接建立和请求到达时才唤醒处理，因此 CPU 时间明显更低。

6 可选任务四：epoll + 协程

6.1 设计思路

任务四保留底层非阻塞 socket 与 epoll 事件等待机制，但用协程封装多轮业务中的等待点。两轮登录业务被写成顺序结构：先发送 `username:`，调用 `co_read()` 等待读取用户名；再发送 `password:`，再次调用 `co_read()` 等待读取密码；最后返回登录结果。

当 `co_read()` 遇到 `EAGAIN` 时，它将当前 fd 注册到 epoll，然后通过 `swapcontext()` 主动让出执行权。事件循环收到该 fd 的可读事件后恢复对应协程。协程上下文保存了程序计数器、栈和局部变量，因此恢复时能从第二次 `co_read()` 内部继续执行，而不是从业务函数开头重新进入。

6.2 完整代码

stage4_coroutine.c

```
1  #define _XOPEN_SOURCE 700
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <stdint.h>
5  #include <string.h>
6  #include <unistd.h>
7  #include <fcntl.h>
8  #include <errno.h>
9  #include <ucontext.h>
10 #include <sys/socket.h>
11 #include <sys/epoll.h>
12 #include <netinet/in.h>
13 #include <arpa/inet.h>
14
15 #ifndef PORT
16 #define PORT 8080
17 #endif
18 #define BACKLOG 1024
19 #define MAX_EVENTS 1024
20 #define MAX_COROUTINES 1024
21 #define STACK_SIZE (64 * 1024)
22 #define BUF_SIZE 256
23
24 static int epoll_fd;
25 static int listen_fd;
26 static ucontext_t sched_ctx;
27 static int current_co = -1;
28
29 typedef struct coroutine_s {
30     int used;
31     int finished;
32     int fd;
33     ucontext_t ctx;
34     char stack[STACK_SIZE];
35 } coroutine_t;
36
37 static coroutine_t coroutines[MAX_COROUTINES];
38
39 static int set_nonblocking(int fd) {
40     int flags = fcntl(fd, F_GETFL, 0);
41     if (flags < 0) return -1;
42     return fcntl(fd, F_SETFL, flags | O_NONBLOCK);
43 }
44
45 static void epoll_mod_read(int fd, int co_id) {
46     struct epoll_event ev;
47     memset(&ev, 0, sizeof(ev));
48     ev.events = EPOLLIN | EPOLLRDHUP;
49     ev.data.u32 = (uint32_t)co_id;
50     if (epoll_ctl(epoll_fd, EPOLL_CTL_MOD, fd, &ev) < 0) {
51         if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, fd, &ev) < 0) {
52             perror("epoll_ctl coroutine");
```

```

53     }
54 }
55 }
56
57 static void coroutine_yield(void) {
58     int id = current_co;
59     current_co = -1;
60     swapcontext(&coroutines[id].ctx, &sched_ctx);
61 }
62
63 static void coroutine_resume(int co_id) {
64     if (co_id < 0 || co_id >= MAX_COROUTINES || !coroutines[co_id].used || coroutines[co_id].finished)
65         return;
66     current_co = co_id;
67     swapcontext(&sched_ctx, &coroutines[co_id].ctx);
68     current_co = -1;
69 }
70
71 static ssize_t co_read(int fd, void *buf, size_t count) {
72     while (1) {
73         ssize_t n = read(fd, buf, count);
74         if (n >= 0)
75             return n;
76         if (errno == EINTR)
77             continue;
78         if (errno == EAGAIN || errno == EWOULDBLOCK) {
79             epoll_mod_read(fd, current_co);
80             coroutine_yield();
81             continue;
82         }
83         return -1;
84     }
85 }
86
87 static void trim_line(char *s) {
88     size_t n = strlen(s);
89     while (n > 0 && (s[n - 1] == '\n' || s[n - 1] == '\r')) {
90         s[n - 1] = '\0';
91         n--;
92     }
93 }
94
95 static void handle_client(uintptr_t arg) {
96     int fd = (int)arg;
97     int id = current_co;
98     char username[BUF_SIZE];
99     char password[BUF_SIZE];
100     const char *banner = "username: ";
101     const char *prompt = "password: ";
102
103     write(fd, banner, strlen(banner));
104     ssize_t n = co_read(fd, username, sizeof(username) - 1);
105     if (n <= 0) goto done;
106     username[n] = '\0';
107     trim_line(username);
108
109     write(fd, prompt, strlen(prompt));
110     n = co_read(fd, password, sizeof(password) - 1);

```

```

111     if (n <= 0) goto done;
112     password[n] = '\0';
113     trim_line(password);
114
115     char resp[512];
116     snprintf(resp, sizeof(resp), "login ok, user=%s, pass=%s\n", username, password);
117     write(fd, resp, strlen(resp));
118     printf("[coroutine] co=%d handled login user=%s password=%s\n", id, username, password);
119     fflush(stdout);
120
121     done:
122     epoll_ctl(epoll_fd, EPOLL_CTL_DEL, fd, NULL);
123     close(fd);
124     coroutines[id].finished = 1;
125     coroutines[id].used = 0;
126     coroutine_yield();
127 }
128
129 static int coroutine_create(int fd) {
130     for (int i = 0; i < MAX_COROUTINES; i++) {
131         if (!coroutines[i].used) {
132             coroutines[i].used = 1;
133             coroutines[i].finished = 0;
134             coroutines[i].fd = fd;
135             getcontext(&coroutines[i].ctx);
136             coroutines[i].ctx.uc_stack.ss_sp = coroutines[i].stack;
137             coroutines[i].ctx.uc_stack.ss_size = sizeof(coroutines[i].stack);
138             coroutines[i].ctx.uc_link = &sched_ctx;
139             makecontext(&coroutines[i].ctx, (void (*)(void))handle_client, 1, (uintptr_t)fd);
140             return i;
141         }
142     }
143     return -1;
144 }
145
146 static int create_listen_socket(void) {
147     int fd = socket(AF_INET, SOCK_STREAM, 0);
148     if (fd < 0) { perror("socket"); exit(1); }
149     int opt = 1;
150     setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
151     struct sockaddr_in addr;
152     memset(&addr, 0, sizeof(addr));
153     addr.sin_family = AF_INET;
154     addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
155     addr.sin_port = htons(PORT);
156     if (bind(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) { perror("bind"); exit(1); }
157     if (listen(fd, BACKLOG) < 0) { perror("listen"); exit(1); }
158     set_nonblocking(fd);
159     return fd;
160 }
161
162 int main(void) {
163     listen_fd = create_listen_socket();
164     epoll_fd = epoll_create1(0);
165     if (epoll_fd < 0) { perror("epoll_create1"); exit(1); }
166     struct epoll_event ev;
167     memset(&ev, 0, sizeof(ev));
168     ev.events = EPOLLIN;

```

```

169     ev.data.u32 = UINT32_MAX;
170     if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, listen_fd, &ev) < 0) { perror("epoll_ctl listen"); exit(1); }
171     printf("stage4 coroutine login server start, port=%d\n", PORT);
172     fflush(stdout);
173
174     struct epoll_event events[MAX_EVENTS];
175     while (1) {
176         int nready = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);
177         if (nready < 0) {
178             if (errno == EINTR) continue;
179             perror("epoll_wait"); break;
180         }
181         for (int i = 0; i < nready; i++) {
182             if (events[i].data.u32 == UINT32_MAX) {
183                 while (1) {
184                     int client_fd = accept(listen_fd, NULL, NULL);
185                     if (client_fd < 0) {
186                         if (errno == EAGAIN || errno == EWOULDBLOCK) break;
187                         if (errno == EINTR) continue;
188                         perror("accept"); break;
189                     }
190                     set_nonblocking(client_fd);
191                     int co_id = coroutine_create(client_fd);
192                     if (co_id < 0) {
193                         fprintf(stderr, "协程槽位已满\n");
194                         close(client_fd);
195                         continue;
196                     }
197                     printf("[coroutine] new client fd=%d -> co=%d\n", client_fd, co_id);
198                     coroutine_resume(co_id);
199                 }
200             } else {
201                 coroutine_resume((int)events[i].data.u32);
202             }
203         }
204     }
205     close(epoll_fd);
206     close(listen_fd);
207     return 0;
208 }

```

6.3 最终运行结果与 Reactor 结构对比

```
可选任务四：epoll + 协程两轮登录运行记录

服务端输出
stage4 coroutine login server start, port=18080
[coroutine] new client fd=5 -> co=0
[coroutine] co=0 handled login user=xsk password=123456

客户端输出
server: username: server: password: server: login ok, user=xsk, pass=123456
```

图 5: 可选任务四运行记录：协程顺序完成两轮登录

Reactor 状态机需要在连接上下文中显式保存当前阶段、用户名、密码、发送进度和接收缓冲区等跨事件数据；每增加一个等待点，都需要增加新的状态分支和状态迁移。协程版本中，用户名和密码可以作为普通局部变量保存在协程栈上，等待点表现为普通函数调用，业务结构更接近同步顺序代码。

7 实验思考题

思考题 1：任务一中 `connect()` 成功为什么不能证明服务端已经成功创建工作线程？客户端还必须检查什么？

作答：`connect()` 成功只说明 TCP 层握手完成，连接进入了服务端监听队列或已被 `accept()` 取出。它不能证明服务端已经为该连接成功创建工作线程，也不能证明应用层已经读取请求并返回响应。在本实验任务一中，服务端可能在 `accept()` 后因为 `pthread_create()` 返回 `EAGAIN` 而关闭连接。因此客户端必须继续发送 HTTP 请求，并检查是否收到有效的 HTTP/1.1 200 OK 响应，才能确认应用层处理成功。

思考题 2：`pthread_create()` 返回 `EAGAIN` 时，为什么操作系统不会在资源释放后自动替程序重新处理已经关闭的连接？

作答：`pthread_create()` 失败是一次系统调用的返回结果，操作系统只负责告知应用程序当前资源不足，并不会保存应用层“应该重新创建线程处理这个连接”的语义。服务端代码已经在失败路径中关闭了客户端 `fd`，TCP 连接状态也发生变化；之后即使资源释放，内核也无法知道应用程序原本想如何处理该连接，更不会自动恢复

已经关闭的文件描述符。因此，重试、排队、限流或返回错误都必须由应用程序显式设计。

思考题 3：任务二只有一个线程，为什么 4000 个客户端不发送数据时仍会消耗接近一个 CPU 核心？

作答：任务二的 socket 虽然是非阻塞的，但程序没有使用事件通知机制，而是不断遍历连接数组，对每个 fd 反复调用 read()。当客户端没有发送数据时，read() 会立即返回 EAGAIN，程序马上继续扫描下一个连接，循环不会休眠。连接越多，无效查询越多，所以即使只有一个线程，CPU 也会持续执行忙轮询循环，表现为接近占满一个核心。

思考题 4：任务三的 epoll_wait(..., -1) 与任务二不断调用非阻塞 read() 在等待方式上有什么本质区别？

作答：任务二是应用程序主动轮询：即使没有连接可读，也要不断调用 read() 并得到 EAGAIN。任务三是事件驱动等待：应用程序将 fd 注册给内核，然后阻塞在 epoll_wait(..., -1)。没有就绪事件时线程睡眠，不消耗持续 CPU；只有内核发现某个 fd 变为可读、关闭或出错时，epoll_wait() 才返回。两者的本质区别在于，任务二由用户态反复查询状态，任务三由内核在状态变化后通知应用程序。

思考题 5：如果使用 Reactor 实现两轮登录，为什么需要显式保存业务状态、用户名和收发进度？业务等待点继续增加时，代码结构会发生什么变化？

作答：Reactor 模型每次只在 fd 就绪时进入事件处理函数，处理函数执行完就返回事件循环。由于一次业务可能被拆成多次事件，例如读取用户名、发送密码提示、读取密码、发送结果，下一次事件到来时程序必须知道上一次处理到哪一步。因此连接上下文需要显式保存业务状态、用户名、密码、发送缓冲区、已发送位置和接收进度等数据。等待点越多，状态枚举和分支越多，代码逐渐变成复杂状态机，可读性和维护性下降。

任务四思考题 1：为什么协程版本中的用户名和密码可以保留为普通局部变量，而 Reactor 版本通常需要把跨事件数据放入连接上下文？

作答：协程挂起时保存的是完整执行上下文，包括当前执行位置、寄存器状态和协程栈。用户名、密码等局部变量位于协程栈中，因此协程恢复后仍然存在。Reactor

版本没有保存调用栈，事件处理函数每次从入口重新执行，局部变量会随函数返回而失效，所以必须把跨事件数据放入堆上的连接上下文。

任务四思考题 2：第二次调用 `co_read()` 后协程挂起。恢复时，CPU 为什么能回到第二次 `co_read()` 内部继续，而不是从 `handle_client()` 开头重新执行？

作答：`co_read()` 挂起时通过上下文切换函数保存了当前协程的 CPU 上下文，包括程序计数器、栈指针和相关寄存器。事件循环再次恢复该协程时，会把这些上下文恢复到 CPU 中，因此程序计数器指向挂起前的位置，栈指针也指向原来的协程栈。于是执行会从第二次 `co_read()` 的挂起点继续，而不是重新调用 `handle_client()`。

8 自选进阶任务：挑战二——多 Reactor 多线程两轮登录服务器

8.1 设计思路

本报告选择“挑战二：多 Reactor 多线程两轮登录服务器”。整体结构采用一个主 Reactor 和多个工作 Reactor：主线程只负责监听端口并 `accept()` 新连接，然后按照轮询策略把连接 `fd` 分发给某个工作线程；每个工作线程拥有独立的 `epoll` 实例、`eventfd` 唤醒 `fd` 和连接队列。主线程将 `fd` 入队后写 `eventfd` 唤醒对应工作线程，工作线程从队列取出 `fd`，注册到自己的 `epoll` 中并处理两轮登录状态机。

关键数据结构包括：`worker_t` 保存工作线程的 `epoll` `fd`、`eventfd` 和待注册 `fd` 队列；`conn_t` 保存单个连接的 `fd`、当前状态、用户名、密码、发送缓冲区和发送进度。业务状态分为 `READ_USER`、`SEND_PASS_PROMPT`、`READ_PASS`、`SEND_RESULT` 等阶段。错误路径统一从 `epoll` 中删除 `fd`、关闭 `fd` 并释放连接对象。

8.2 代码实现

`multi_reactor_login.c`

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdint.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <fcntl.h>
7 #include <errno.h>
8 #include <pthread.h>
```



```

9  #include <sys/socket.h>
10 #include <sys/epoll.h>
11 #include <sys/eventfd.h>
12 #include <sys/resource.h>
13 #include <netinet/in.h>
14 #include <arpa/inet.h>
15
16 #ifndef PORT
17 #define PORT 8080
18 #endif
19 #define WORKERS 4
20 #define BACKLOG 4096
21 #define MAX_EVENTS 1024
22 #define BUF_SIZE 512
23 #define QUEUE_SIZE 8192
24 #define FILE_LIMIT 10000
25
26 typedef enum { READ_USER, SEND_PASS_PROMPT, READ_PASS, SEND_RESULT, CLOSING } state_t;
27
28 typedef struct conn_s {
29     int fd;
30     state_t state;
31     char username[128];
32     char password[128];
33     char recv_buf[BUF_SIZE];
34     char send_buf[BUF_SIZE];
35     size_t send_len;
36     size_t send_pos;
37 } conn_t;
38
39 typedef struct worker_s {
40     int id;
41     int epfd;
42     int event_fd;
43     pthread_t tid;
44     int queue[QUEUE_SIZE];
45     int head, tail;
46     pthread_mutex_t lock;
47     long handled;
48 } worker_t;
49
50 static worker_t workers[WORKERS];
51
52 static int set_nonblocking(int fd) {
53     int flags = fcntl(fd, F_GETFL, 0);
54     if (flags < 0) return -1;
55     return fcntl(fd, F_SETFL, flags | O_NONBLOCK);
56 }
57
58 static void raise_fd_limit(void) {
59     struct rlimit limit;
60     if (getrlimit(RLIMIT_NOFILE, &limit) < 0) return;
61     rlim_t target = FILE_LIMIT;
62     if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max) target = limit.rlim_max;
63     limit.rlim_cur = target;
64     setrlimit(RLIMIT_NOFILE, &limit);
65 }
66

```

```

67 static void trim_line(char *s) {
68     size_t n = strlen(s);
69     while (n > 0 && (s[n-1] == '\n' || s[n-1] == '\r')) { s[--n] = '\0'; }
70 }
71
72 static int queue_push(worker_t *w, int fd) {
73     pthread_mutex_lock(&w->lock);
74     int next = (w->tail + 1) % QUEUE_SIZE;
75     if (next == w->head) {
76         pthread_mutex_unlock(&w->lock);
77         return -1;
78     }
79     w->queue[w->tail] = fd;
80     w->tail = next;
81     pthread_mutex_unlock(&w->lock);
82     uint64_t one = 1;
83     write(w->event_fd, &one, sizeof(one));
84     return 0;
85 }
86
87 static int queue_pop(worker_t *w) {
88     pthread_mutex_lock(&w->lock);
89     if (w->head == w->tail) { pthread_mutex_unlock(&w->lock); return -1; }
90     int fd = w->queue[w->head];
91     w->head = (w->head + 1) % QUEUE_SIZE;
92     pthread_mutex_unlock(&w->lock);
93     return fd;
94 }
95
96 static void epoll_add(worker_t *w, int fd, uint32_t events, conn_t *c) {
97     struct epoll_event ev;
98     memset(&ev, 0, sizeof(ev));
99     ev.events = events | EPOLLRDHUP;
100     ev.data.ptr = c;
101     if (epoll_ctl(w->epfd, EPOLL_CTL_ADD, fd, &ev) < 0) perror("epoll_ctl ADD");
102 }
103
104 static void epoll_mod(worker_t *w, conn_t *c, uint32_t events) {
105     struct epoll_event ev;
106     memset(&ev, 0, sizeof(ev));
107     ev.events = events | EPOLLRDHUP;
108     ev.data.ptr = c;
109     if (epoll_ctl(w->epfd, EPOLL_CTL_MOD, c->fd, &ev) < 0) perror("epoll_ctl MOD");
110 }
111
112 static void conn_close(worker_t *w, conn_t *c) {
113     if (!c) return;
114     epoll_ctl(w->epfd, EPOLL_CTL_DEL, c->fd, NULL);
115     close(c->fd);
116     free(c);
117     w->handled++;
118 }
119
120 static void handle_read(worker_t *w, conn_t *c) {
121     ssize_t n = read(c->fd, c->recv_buf, sizeof(c->recv_buf)-1);
122     if (n <= 0) { if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) return; conn_close(w, c); return;
123     ↵ }
124     c->recv_buf[n] = '\0'; trim_line(c->recv_buf);

```

```

124     if (c->state == READ_USER) {
125         snprintf(c->username, sizeof(c->username), "%s", c->recv_buf);
126         snprintf(c->send_buf, sizeof(c->send_buf), "password: ");
127         c->send_len = strlen(c->send_buf); c->send_pos = 0;
128         c->state = SEND_PASS_PROMPT;
129         epoll_mod(w, c, EPOLLOUT);
130     } else if (c->state == READ_PASS) {
131         snprintf(c->password, sizeof(c->password), "%s", c->recv_buf);
132         snprintf(c->send_buf, sizeof(c->send_buf), "login ok, user=%s, pass=%s\n", c->username,
133             ↪ c->password);
134         c->send_len = strlen(c->send_buf); c->send_pos = 0;
135         c->state = SEND_RESULT;
136         epoll_mod(w, c, EPOLLOUT);
137     }
138 }
139
140 static void handle_write(worker_t *w, conn_t *c) {
141     while (c->send_pos < c->send_len) {
142         ssize_t n = write(c->fd, c->send_buf + c->send_pos, c->send_len - c->send_pos);
143         if (n > 0) c->send_pos += (size_t)n;
144         else if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) return;
145         else { conn_close(w, c); return; }
146     }
147     if (c->state == SEND_PASS_PROMPT) {
148         c->state = READ_PASS;
149         epoll_mod(w, c, EPOLLIN);
150     } else if (c->state == SEND_RESULT) {
151         conn_close(w, c);
152     }
153 }
154
155 static void worker_register_fd(worker_t *w, int fd) {
156     set_nonblocking(fd);
157     conn_t *c = calloc(1, sizeof(*c));
158     if (!c) { close(fd); return; }
159     c->fd = fd;
160     c->state = READ_USER;
161     const char *hello = "username: ";
162     (void)write(fd, hello, strlen(hello));
163     epoll_add(w, fd, EPOLLIN, c);
164 }
165
166 static void *worker_loop(void *arg) {
167     worker_t *w = arg;
168     struct epoll_event events[MAX_EVENTS];
169     while (1) {
170         int n = epoll_wait(w->epfd, events, MAX_EVENTS, -1);
171         if (n < 0) { if (errno == EINTR) continue; perror("worker epoll_wait"); break; }
172         for (int i = 0; i < n; i++) {
173             if (events[i].data.ptr == NULL) {
174                 uint64_t tmp;
175                 while (read(w->event_fd, &tmp, sizeof(tmp)) > 0) {}
176                 int fd;
177                 while ((fd = queue_pop(w)) >= 0) {
178                     printf("worker %d take fd=%d\n", w->id, fd);
179                     worker_register_fd(w, fd);
180                 }
181             } else {

```

```

181         conn_t *c = events[i].data.ptr;
182         if (events[i].events & (EPOLLHUP | EPOLLERR | EPOLLRDHUP)) { conn_close(w, c); continue; }
183         if (events[i].events & EPOLLIN) handle_read(w, c);
184         else if (events[i].events & EPOLLOUT) handle_write(w, c);
185     }
186 }
187 }
188 return NULL;
189 }
190
191 static int create_listen_socket(void) {
192     int fd = socket(AF_INET, SOCK_STREAM, 0);
193     if (fd < 0) { perror("socket"); exit(1); }
194     int opt = 1;
195     setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
196     struct sockaddr_in addr;
197     memset(&addr, 0, sizeof(addr));
198     addr.sin_family = AF_INET;
199     addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
200     addr.sin_port = htons(PORT);
201     if (bind(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) { perror("bind"); exit(1); }
202     if (listen(fd, BACKLOG) < 0) { perror("listen"); exit(1); }
203     return fd;
204 }
205
206 int main(void) {
207     raise_fd_limit();
208     for (int i = 0; i < WORKERS; i++) {
209         workers[i].id = i;
210         workers[i].epfd = epoll_create1(0);
211         workers[i].event_fd = eventfd(0, EFD_NONBLOCK);
212         pthread_mutex_init(&workers[i].lock, NULL);
213         struct epoll_event ev;
214         memset(&ev, 0, sizeof(ev));
215         ev.events = EPOLLIN;
216         ev.data.ptr = NULL;
217         epoll_ctl(workers[i].epfd, EPOLL_CTL_ADD, workers[i].event_fd, &ev);
218         pthread_create(&workers[i].tid, NULL, worker_loop, &workers[i]);
219     }
220     int listen_fd = create_listen_socket();
221     printf("multi-reactor login server start, port=%d, workers=%d\n", PORT, WORKERS);
222     fflush(stdout);
223     long accepted = 0;
224     while (1) {
225         int fd = accept(listen_fd, NULL, NULL);
226         if (fd < 0) { if (errno == EINTR) continue; perror("accept"); break; }
227         worker_t *w = &workers[accepted % WORKERS];
228         printf("main accept fd=%d -> worker %d\n", fd, w->id);
229         if (queue_push(w, fd) < 0) close(fd);
230         accepted++;
231         if (accepted % 100 == 0) printf("accepted=%ld\n", accepted);
232     }
233     close(listen_fd);
234     return 0;
235 }

```

并发登录测试客户端 **login_many_client.c**

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/resource.h>
6 #include <sys/socket.h>
7 #include <arpa/inet.h>
8
9 static void raise_fd_limit(int count) {
10     struct rlimit limit;
11     if (getrlimit(RLIMIT_NOFILE, &limit) < 0) return;
12     rlim_t target = count + 100;
13     if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max) target = limit.rlim_max;
14     limit.rlim_cur = target;
15     setrlimit(RLIMIT_NOFILE, &limit);
16 }
17
18 int main(int argc, char **argv) {
19     int n = argc > 1 ? atoi(argv[1]) : 200;
20     raise_fd_limit(n);
21     int ok = 0;
22     for (int i = 0; i < n; i++) {
23         int fd = socket(AF_INET, SOCK_STREAM, 0);
24         if (fd < 0) break;
25         struct sockaddr_in addr;
26         memset(&addr, 0, sizeof(addr));
27         addr.sin_family = AF_INET;
28         addr.sin_port = htons(PORT);
29         inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);
30         if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) { close(fd); break; }
31         char buf[512];
32         ssize_t r = read(fd, buf, sizeof(buf)-1);
33         if (r <= 0) { close(fd); continue; }
34         dprintf(fd, "user%d\n", i);
35         r = read(fd, buf, sizeof(buf)-1);
36         if (r <= 0) { close(fd); continue; }
37         dprintf(fd, "pass%d\n", i);
38         r = read(fd, buf, sizeof(buf)-1);
39         if (r > 0) ok++;
40         close(fd);
41     }
42     printf("login_many result: target=%d ok=%d\n", n, ok);
43     return 0;
44 }
```

8.3 最终结果或运行输出

自选进阶：多 Reactor 多线程两轮登录服务器运行记录

服务端输出 (节选)

```
multi-reactor login server start, port=18080, workers=4
main accept fd=12 -> worker 0
worker 0 take fd=12
main accept fd=12 -> worker 1
worker 1 take fd=12
main accept fd=12 -> worker 2
worker 2 take fd=12
main accept fd=12 -> worker 3
worker 3 take fd=12
main accept fd=12 -> worker 0
worker 0 take fd=12
main accept fd=12 -> worker 1
worker 1 take fd=12
main accept fd=12 -> worker 2
worker 2 take fd=12
main accept fd=12 -> worker 3
worker 3 take fd=12
main accept fd=12 -> worker 0
worker 0 take fd=12
main accept fd=12 -> worker 1
... (中间省略) ...
worker 2 take fd=12
main accept fd=12 -> worker 3
worker 3 take fd=12
main accept fd=12 -> worker 0
worker 0 take fd=12
main accept fd=12 -> worker 1
worker 1 take fd=12
main accept fd=12 -> worker 2
worker 2 take fd=12
main accept fd=12 -> worker 3
worker 3 take fd=12
main accept fd=12 -> worker 0
worker 0 take fd=12
main accept fd=12 -> worker 1
worker 1 take fd=12
main accept fd=12 -> worker 2
worker 2 take fd=12
main accept fd=12 -> worker 3
accepted=200
worker 3 take fd=12
```

客户端并发测试输出

```
login_many result: target=200 ok=200
```

图 6: 多 Reactor 多线程两轮登录服务器运行记录

测试结果显示，主线程按轮询方式将新连接分发给 4 个 worker；worker 被 `eventfd` 唤醒后取出 `fd` 并注册到自己的 `epoll`。并发测试客户端共发起 200 次两轮登录，最终 `ok=200`，说明用户名/密码两轮读写、跨事件状态保存、`fd` 分发和资源释放均正常工作。该方案没有为每个连接创建线程，而是由多个 Reactor 线程共同管理连接，满足“多 Reactor 多线程两轮登录服务器”的进阶要求。

9 实验总结

本实验从多线程阻塞 I/O 出发，观察到线程资源会随慢连接数量线性增长，并在地址空间限制下触发 `pthread_create()` 失败。非阻塞忙轮询虽然消除了线程爆炸，却将等待成本转化为用户态反复扫描连接数组，导致 CPU 时间明显升高。`epoll` 将等待

交给内核完成，应用程序只处理就绪 fd，显著降低了空闲连接保持阶段的 CPU 消耗。进一步的协程实验说明，协程不改变底层 `epoll` 的同步就绪通知本质，但能保存执行位置和局部变量，使多轮业务代码恢复为顺序结构。最后的多 Reactor 扩展则展示了在多核场景下将连接分发给多个独立事件循环的基本方法。